

Deploy BOWL on a VPS (nginx + gunicorn)

This guide moves the combined hub + three league sites off shared hosting (e.g. PythonAnywhere) onto a small Linux VPS with **nginx** in front and **gunicorn** running `wsgi.application`. The app layout is unchanged: `/`, `/bowl-historical/`, `/bowl-fantasy/`, `/bowl-cap/`.

For CSV import and update cycles after the server exists, keep using [DATA-UPDATE.md](#) and [UPDATE-NESTED-SERVER.md](#) (steps 6–9).

1. VPS sizing (starting point)

Resource	Suggestion
RAM	2 GB minimum (3 league SQLite DBs + site DB + gunicorn workers)
CPU	1–2 vCPU
Disk	20 GB+ (CSV imports, <code>instance/*.db</code> , <code>instance/league_json_cache/</code> , static assets)
OS	Ubuntu 22.04 or 24.04 LTS

2. Server packages

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nginx git
```

Optional: `certbot python3-certbot-nginx` for Let's Encrypt.

3. App user and checkout

```
sudo adduser --disabled-password bowl
sudo mkdir -p /srv/bowl
sudo chown bowl:bowl /srv/bowl
sudo -u bowl git clone https://github.com/YOUR_ORG/Boys-Of-Winter-League.git
/srv/bowl/app
cd /srv/bowl/app
sudo -u bowl python3 -m venv .venv
sudo -u bowl .venv/bin/pip install -r requirements.txt
sudo -u bowl .venv/bin/pip install gunicorn
```

Copy production env (never commit secrets):

```
sudo -u bowl cp .env.example .env
# edit /srv/bowl/app/.env – see section 5
```

Ensure live databases and CSVs exist under `instance/` and `data/imports/raw/` (rsync or your existing deploy scripts).

4. Environment variables (VPS)

In `/srv/bowl/app/.env` (loaded by the app via `python-dotenv` where configured, or export from systemd):

```
FLASK_DEBUG=0
SECRET_KEY=<long-random-string>
SITE_PUBLIC_BASE_URL=https://www.bowlhockey.com

# Recommended on a dedicated VPS (pre-build homepage JSON after worker start /
import)
LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1

# Optional: paint leaders + standings in HTML from cache (no cold build on home HTML)
HOMEPAGE_SSR_LEADERS_STANDINGS=1

# Site DB – keep MySQL if you already use PA MySQL, or SQLite on VPS:
# SITE_DATABASE_URL=mysql+pymysql://...
```

Leave `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=0` on PythonAnywhere if you stay on PA (many workers × three leagues can spike SQLite I/O on reload).

5. systemd unit (gunicorn)

Create `/etc/systemd/system/bowl-web.service` :

```
[Unit]
Description=BOWL combined WSGI (hub + leagues)
After=network.target

[Service]
User=bowl
Group=bowl
WorkingDirectory=/srv/bowl/app
EnvironmentFile=/srv/bowl/app/.env
Environment=PYTHONPATH=/srv/bowl/app
ExecStart=/srv/bowl/app/.venv/bin/gunicorn \
  --bind 127.0.0.1:8000 \
  --workers 3 \
  --threads 2 \
  --timeout 120 \
  --worker-class gthread \
  --access-logfile - \
  --error-logfile - \
  wsgi:application
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Notes:

- `wsgi:application` is the combined DispatcherMiddleware app (same as PythonAnywhere).
- `--workers 3` : one process can lazy-load each league; 3–4 workers is a reasonable start on 2 GB RAM. Increase only if CPU is idle and RAM allows.
- `--timeout 120` : cold homepage cache builds can be slow; warm cache keeps requests fast.
- First request to a league after a worker fork still pays lazy `create_app` (see `wsgi.py`); warming mitigates JSON endpoints, not that one-time init.

Enable:

```
sudo systemctl daemon-reload
sudo systemctl enable bowl-web
sudo systemctl start bowl-web
sudo systemctl status bowl-web
```

6. nginx site

Create `/etc/nginx/sites-available/bowl`:

```
upstream bowl_app {
    server 127.0.0.1:8000;
    keepalive 8;
}

server {
    listen 80;
    server_name www.bowlhockey.com bowlhockey.com;

    client_max_body_size 20M;

    # Long-cache static assets (combined static middleware + app/static)
    location ~* ^/(bowl-historical|bowl-fantasy|bowl-cap|bowl-formula|bowl-
demolition)?/static/ {
        alias /srv/bowl/app/app/static/;
        try_files $uri @app;
        expires 7d;
        add_header Cache-Control "public, max-age=604800";
    }

    location / {
        proxy_pass http://bowl_app;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }

    location @app {
        proxy_pass http://bowl_app;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Enable and test:

```
sudo ln -sf /etc/nginx/sites-available/bowl /etc/nginx/sites-enabled/bowl
sudo nginx -t
sudo systemctl reload nginx
```

TLS: `sudo certbot --nginx -d www.bowlhockey.com -d bowlhockey.com`

Path layout: nginx must forward the **full** path (`/bowl-fantasy/...`) to gunicorn without stripping the league prefix. Do **not** set `SCRIPT_NAME` unless you use an extra URL prefix above the league slugs (see [UPDATE-NESTED-SERVER.md](#)).

Discord interactions and hub routes use `SITE_PUBLIC_BASE_URL` at the domain root (e.g. `https://www.bowlhockey.com/api/discord/interactions`), not under a league prefix.

7. Post-deploy cache warm (recommended)

After `systemctl restart bowl-web` or a DB upload:

```
curl -s -o /dev/null -w "%{http_code} summary\n" \
  "https://www.bowlhockey.com/bowl-fantasy/api/homepage/summary?segment=rs"
curl -s -o /dev/null -w "%{http_code} odds\n" \
  "https://www.bowlhockey.com/bowl-fantasy/api/homepage/postseason-odds"
# Repeat for bowl-historical and bowl-cap
```

Or rely on `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1` plus import-time warm in `rebuild.py`.

8. uWSGI instead of gunicorn (optional)

If you prefer uWSGI:

```
pip install uwsgi
```

Example ini:

```
[uwsgi]
module = wsgi:application
chdir = /srv/bowl/app
virtualenv = /srv/bowl/app/.venv
master = true
processes = 3
threads = 2
socket = 127.0.0.1:8000
chmod-socket = 660
vacuum = true
die-on-term = true
harakiri = 120
env = PYTHONPATH=/srv/bowl/app
```

Point nginx `proxy_pass` at the socket or HTTP listener. Gunicorn is usually simpler for Flask; both work with the same `wsgi:application`.

9. Discord bot and cron on the VPS

Run the league Discord bot as a **separate** systemd service (same repo, same `.env`), pointing `DISCORD_BOT_LEAGUE_BASE_URLS` at your public URLs.

Keep using `scripts/run_site_update.py` from your dev machine or a CI runner; on the VPS you only need `git pull` / `rsync` DBs, imports, and `systemctl restart bowl-web`.

10. Migration checklist from PythonAnywhere

1. Copy `instance/*.db`, `instance/site_membership.db`, and `instance/league_json_cache/` if present.
 2. Copy `data/imports/raw/**` and `app/static/**` (logos, headshots).
 3. Set `SITE_DATABASE_URL` if using remote MySQL; otherwise migrate site DB to VPS SQLite/MySQL.
 4. Deploy code, install deps, configure systemd + nginx.
 5. Set `LEAGUE_JSON_CACHE_WARM_ON_STARTUP=1`, optionally `HOMEPAGE_SSR_LEADERS_STANDINGS=1`.
 6. Restart web, run `curl warm` (section 7), smoke-test hub + each league + search.
 7. Update DNS A/AAAA to the VPS; keep PA up until TTL expires if doing a cutover.
-

Quick reference

Component	Value
WSGI entry	<code>wsgi:application</code>
Working directory	project root (contains <code>wsgi.py</code>)
JSON cache dir	<code>instance/league_json_cache/</code>
League DBs	<code>instance/bowl-historical.db</code> , <code>bowl-fantasy.db</code> , <code>bowl-cap.db</code>
Site DB	<code>instance/site_membership.db</code> or <code>SITE_DATABASE_URL</code>